SWE-Bench Mobile: Can Large Language Model Agents Develop Industry-Level Mobile Applications?

Muxin Tian*
University of Toronto
Toronto, Canada
Xiaohongshu Inc.
Shanghai, China
murphy.tian@mail.utoronto.ca

Zhe Wang*
Xiaohongshu Inc.
Shanghai, China
wangzhe101@xiaohongshu.com

Zhenwei Tang
University of Toronto
Toronto, Canada
josephtang@cs.toronto.edu

Blair Yang
Coolwei AI Lab
Beijing, China
yangliwei@coolwei.com

Kunlun Zhu
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
kunlunz2@illinois.edu

Honghua Dong
University of Toronto
Toronto, Canada
honghuad@cs.toronto.edu

Hanchen Li
University of California, Berkeley
Berkeley, California, USA
lihanc@berkeley.edu

Xinni Xie
Xiaohongshu Inc.
Shanghai, China
xiexinni@xiaohongshu.com

Guangjing Wang
Xiaohongshu Inc.
Shanghai, China
wangguangjing@xiaohongshu.com

Jiaxuan You†
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
jiaxuan@illinois.edu

Abstract

Can large language model agents develop industry-level mobile applications? We introduce **SWE-Bench Mobile**, a benchmark for evaluating coding agents on realistic software engineering tasks derived from a production iOS codebase. Unlike existing benchmarks that focus on isolated problems or bug fixes, SWE-Bench Mobile captures the full complexity of industrial development: multi-modal inputs (PRDs and Figma designs), a large-scale mixed Swift/Objective-C codebase, and comprehensive test suites. We evaluate 22 agent-model configurations across four coding agents—three commercial (Cursor, Codex, Claude Code) and one open-source (OpenCode)—and find that even the best configurations achieve only 12% task success rate. Our analysis reveals that (1) agent design matters as much as model capability—the same model shows up to 6× performance gap across agents, (2) commercial agents consistently outperform open-source alternatives, and (3) simple "Defensive Programming" prompts raise the test pass rate by 7.4 points over the baseline, while overly complex prompts hurt task success. These findings highlight a significant gap between current agent capabilities and industrial requirements, while providing actionable insights for practitioners and researchers. We release SWE-Bench Mobile as a *hosted benchmark challenge* to prevent data contamination and ensure fair evaluation. The public leaderboard is available at https://swebenchmobile.com.

CCS Concepts

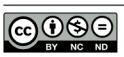
• **Computing methodologies** → **Artificial intelligence**; • **Software and its engineering** → *Software verification and validation.*

Keywords

Large Language Models; Software Engineering Agents; Mobile Development Benchmark

ACM Reference Format:
Muxin Tian, Zhe Wang, Zhenwei Tang, Blair Yang, Kunlun Zhu, Honghua Dong, Hanchen Li, Xinni Xie, Guangjing Wang, and Jiaxuan You. 2026. SWE-Bench Mobile: Can Large Language Model Agents Develop Industry-Level Mobile Applications?. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3770855.3818488

*Equal contribution.
†Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
KDD 2026, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3818488

1 Introduction

Large language models (LLMs) have enabled a new generation of autonomous coding agents that can understand requirements, navigate codebases, and implement features with minimal human

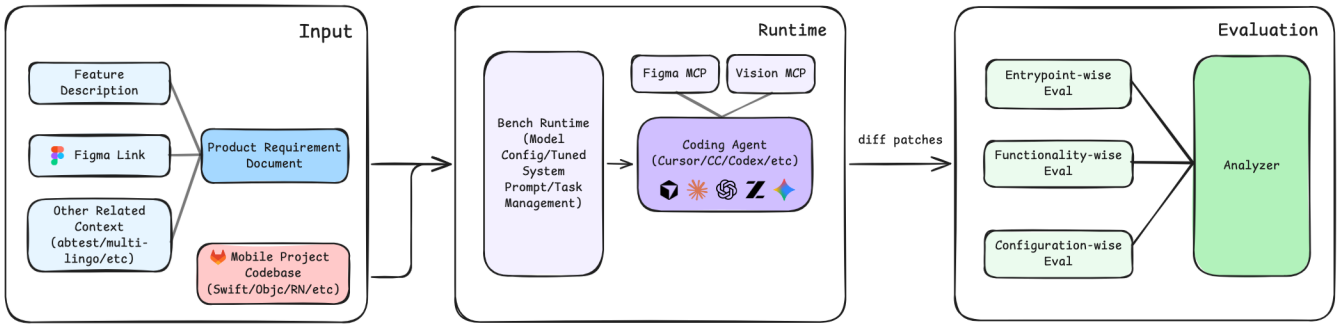


Figure 1: Overview of the SWE-Bench Mobile pipeline. (1) Agents receive multi-modal inputs including a Product Requirement Document (PRD), Figma design, and a large-scale Swift/Objective-C codebase. (2) The agent navigates the codebase, plans the implementation, and generates code. (3) The output is a Git patch that is applied and evaluated against a comprehensive test suite.

intervention. Commercial systems like GitHub Copilot, Cursor, and Claude Code have achieved impressive results on existing benchmarks, raising a critical question: *Can these agents handle the complexity of real-world, industry-level mobile software development?*

Answering this question requires a comprehensive evaluation that faithfully captures professional software engineering. However, existing benchmarks have significant limitations. HumanEval [3] and MBPP [2] evaluate isolated algorithmic problems far removed from industrial practice. SWE-Bench [7] advances the field by using real GitHub issues, but still falls short of industrial realism: it focuses on bug fixes rather than feature development, uses text-only inputs without design specifications, typically involves small localized changes to 1-2 files, and concentrates on Python which is well-represented in training data. Recent work like SWE-Bench Pro [5] addresses some limitations by introducing longer-horizon tasks, but still lacks multi-modal inputs and focuses exclusively on Python.

In professional software development, engineers participate in a structured workflow that goes far beyond writing code. They interpret Product Requirement Documents (PRDs) that specify what to build and why. They translate visual designs from tools like Figma into implementation decisions about layout and interaction. They navigate large codebases—often hundreds of thousands of lines—to find relevant files and understand existing patterns. They make coordinated changes across multiple modules while maintaining consistency. And they ensure their implementations handle edge cases and pass comprehensive tests. A benchmark claiming to evaluate "industry-level" capabilities must test all of these aspects.

We focus on mobile application development not merely for language diversity, but because it represents a distinct and critical paradigm in software engineering that remains unexplored by current benchmarks. Unlike server-side logic (e.g., Python scripts), mobile development introduces unique challenges for AI agents: (1) **Multi-modal Dependency:** Implementation is strictly guided by visual designs (Figma) and user interactions, requiring agents to perform visually-grounded program synthesis rather than just text-to-code generation. (2) **Event-Driven Complexity:** Mobile apps are stateful systems that must handle asynchronous user events, network changes, and strict OS lifecycle callbacks, challenging agents' ability to model dynamic system states. (3) **Client-Side Constraints:** Development occurs within framework-heavy environments (e.g., iOS SDK) with rapid iterations, testing generalization to domain-specific APIs.

We introduce **SWE-Bench Mobile**, a benchmark for evaluating coding agents on industry-level mobile application development. SWE-Bench Mobile is constructed from real development artifacts at a major technology company, comprising 50 authentic tasks derived from actual product requirements. Each task combines multi-modal inputs—PRDs, Figma designs, and a large-scale mixed Swift/Objective-C production codebase—with comprehensive evaluation through 449 human-verified test cases.

Contributions.

(1) We introduce SWE-Bench Mobile, the first benchmark combining PRDs, Figma designs, and a large-scale codebase to capture the full complexity of industrial software development.
(2) We evaluate 22 agent-model configurations across four coding agents (three commercial, one open-source), with detailed analysis of performance, cost, and robustness.
(3) We systematically categorize agent failures, finding that 54% stem from missing feature flags—a production practice unfamiliar to agents—followed by missing data models (22%) and incomplete file coverage (11–15%).
(4) We provide actionable insights: agent design matters as much as model capability (up to 6× performance gap for the same model), commercial agents outperform open-source ones, simple prompts outperform complex ones, and cost-effective configurations exist.

To strictly preserve the integrity of the evaluation and respect the proprietary nature of the production codebase, we adopt a *hosted evaluation paradigm*. Unlike static datasets that are prone to data contamination in future model training sets, our *held-out private test set* ensures that agents are evaluated on truly unseen industrial tasks. We maintain a public leaderboard at https://swebenchmobile.com to report hosted evaluation results.

Our evaluation reveals a significant gap between current capabilities and industrial requirements. The best configuration achieves only 12% task success rate, with most failures due to incomplete

implementations. The same model (Opus 4.5) achieves 12% on Cursor but only 2% on OpenCode—a 6× gap—demonstrating that agent scaffolding matters as much as model capability. Commercial agents consistently outperform the open-source OpenCode, whose best result (8% with GLM-4.6) trails the best commercial result (12%) by 4 percentage points. Success drops from 18% for simple tasks requiring 1-2 files to just 2% for complex tasks requiring 7+ files, indicating agents struggle with cross-file reasoning. These findings suggest that while coding agents show promise for simple tasks, substantial improvements in requirement understanding, multi-modal reasoning, and codebase navigation are needed for reliable industry-level development.

2 SWE-Bench Mobile

SWE-Bench Mobile is a benchmark designed to evaluate coding agents on industry-level mobile application development. Unlike existing benchmarks that focus on isolated coding problems or bug fixes, SWE-Bench Mobile captures the full complexity of professional software engineering: multi-modal inputs, large codebases, and comprehensive testing. Figure 1 illustrates the overall benchmark pipeline.

2.1 Problem Formulation

Each benchmark instance is represented as a triplet:

$$\mathcal{T} = (\mathcal{I}, \mathcal{O}, \mathcal{E}),$$

where $\mathcal{I}$ is the input context, $\mathcal{O}$ is the expected output, and $\mathcal{E}$ is the evaluation configuration.

Input ($\mathcal{I}$). The input context mimics a typical developer's starting point for a new feature. It consists of three main components (see Figure 2). First, a **Product Requirement Document (PRD)** describes the feature goal, user story, acceptance criteria, and constraints. These PRDs are derived from actual product requirements at XiaoHongShu Inc., a major social media platform with over 300 million monthly active users, follow standard industrial conventions [1], and have an average length of 450 words, requiring agents to parse natural language specifications. Second, 70% of tasks include a **Figma Design** specification, containing component layout, typography, and visual details that the agent must translate into code. Finally, the agent is provided with the **XiaoHongShu production codebase**, a Git repository snapshot containing approximately 500,000 lines of Swift/Objective-C code across thousands of files. This large-scale context forces the agent to perform retrieval and navigation, rather than just code generation.

Output ($\mathcal{O}$). The expected output is a unified diff patch that, when applied to the codebase, implements the feature described in the PRD. This format matches the standard pull request workflow used in industry.

Evaluation ($\mathcal{E}$). Each task is paired with a task-specific `pytest` suite (9.1 tests per task on average) that evaluates the generated *patch* directly. Concretely, tests operate on the unified diff *text* without compiling or running the iOS application, and therefore avoid build-time overhead and simulator/device nondeterminism. This patch-level evaluation is designed to verify the *presence* of necessary UI-facing edits (e.g., view construction, layout logic) and data/logic edits (e.g., control-flow, state updates), while remaining tolerant to superficial variability such as identifier naming, refactoring style, and minor structural reorganization.

Table 1: SWE-Bench Mobile dataset statistics.

Metric	Value
Total Tasks	50
Total Test Cases	449
Avg. Test Cases per Task	9.1
Tasks with Figma Design	35 (70%)
Tasks with Reference Images	46 (92%)
Avg. PRD Length (words)	450
Codebase Size	~500K LoC (~5GB)
Programming Language	Swift/Objective-C (iOS)
Avg. Files Modified per Task	4.2

2.2 Design Principles

SWE-Bench Mobile is constructed under guiding principles to ensure relevance to professional software engineering. **End-to-End Realism** is paramount; tasks span the full engineering process from PRD to testing, preserving real-world dependencies and incomplete specifications. Unlike synthetic benchmarks, our tasks come from actual product development cycles. **Multi-Modal Reasoning** is required, as agents must jointly interpret textual requirements (PRD), visual designs (Figma), and structured code. **Diverse Coverage** ensures robustness, with tasks covering multiple categories (Figure 3) and difficulty levels, from simple UI adjustments to complex architectural refactoring. Finally, by focusing on Swift/Objective-C, an **Under-Represented Language** in LLM training data compared to Python or JavaScript, SWE-Bench Mobile serves as a challenging test of an agent's ability to generalize to less familiar syntax and frameworks.

2.3 Dataset Statistics

Table 1 summarizes the key statistics of SWE-Bench Mobile. The benchmark consists of 50 tasks with 449 total test cases. The majority of tasks (70%) include Figma designs, and 92% include reference images, highlighting the multi-modal nature of the dataset. The average PRD length is 450 words, providing substantial context. The codebase scale is significant, with the repository size reaching approximately 5GB.

2.4 Task Construction

Source. Tasks are derived from real product requirements at XiaoHongShu Inc., a leading social media platform in China with over 300 million monthly active users. Each task represents a feature that was actually implemented by XiaoHongShu engineers in the production iOS application, ensuring realistic complexity and scope. Unlike existing benchmarks that use synthetic problems or isolated bug fixes from open-source repositories, our tasks capture the full complexity of feature development in a commercial mobile application: multi-file changes, UI/UX implementation from design specs, integration with existing business logic, and handling of edge

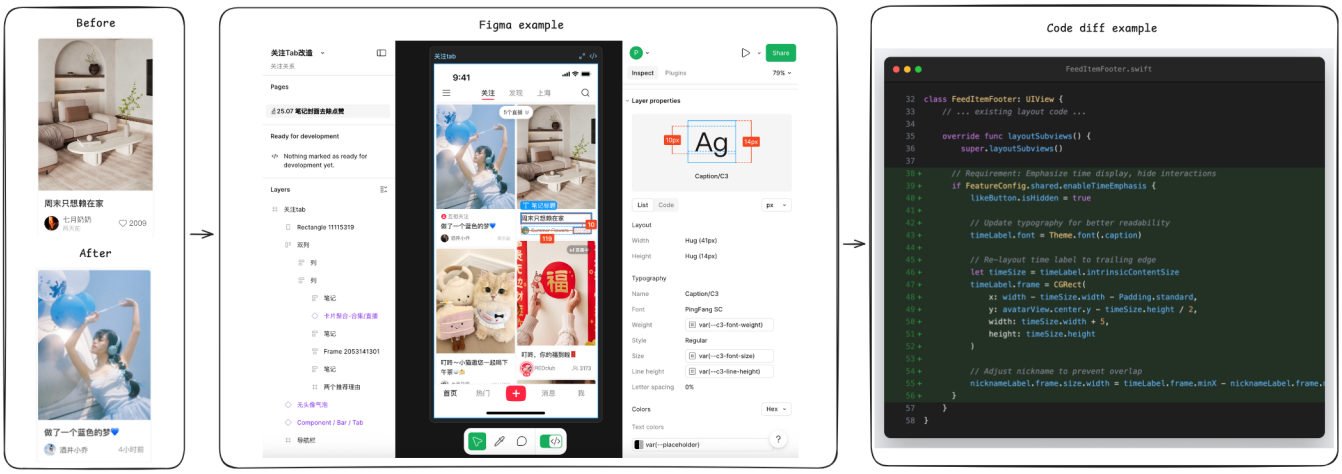


Figure 2: A concrete example of a SWE-Bench Mobile task (Task 056). The agent must interpret the PRD requirements (replace interaction button with publish time label) and visual design (Figma), locate the relevant files in the codebase (FeedItemFooter.swift), and implement the changes while handling edge cases and feature configuration.

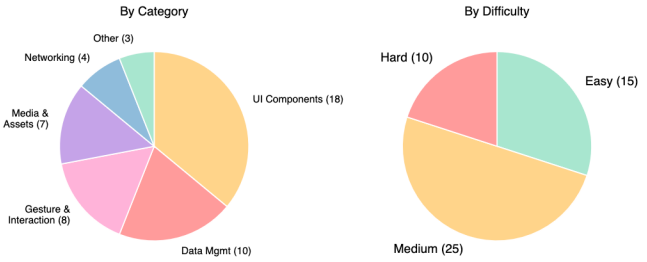


Figure 3: Task distribution by category (left) and difficulty (right). Each slice is labeled with its task count. UI Components (36%) dominate the benchmark, while performance drops sharply from Easy (18.5% pass) to Hard (5.8% pass).

cases and feature flags. This industry-sourced approach ensures that our benchmark reflects the actual challenges faced by software engineers in production environments. **Quality Control.** Each task undergoes a rigorous multi-stage review process. First, the PRDs are reviewed to ensure requirements are clear and self-contained. Next, comprehensive test suites are designed to verify both correctness and quality. Finally, we perform human validation to verify that the reference implementation passes all tests.

Difficulty Calibration. Tasks are labeled by implementation complexity based on several factors: the number of files to modify (1-2 for Easy, 3-5 for Medium, 6+ for Hard), the lines of code changed (<50 for Easy, 50-150 for Medium, >150 for Hard), and the architectural complexity, distinguishing between localized changes and cross-module refactoring.

2.5 Evaluation Pipeline

Unlike traditional code benchmarks that rely solely on unit tests, SWE-Bench Mobile performs comprehensive verification through a multi-step pipeline.

Patch-to-Task Routing. SWE-Bench Mobile evaluates submissions as unified diff patches and associates each patch with a specific task. This routing step ensures that each submission is evaluated under the task's PRD-defined intent and its corresponding test suite, while keeping the evaluation independent of repository checkout, compilation, or runtime execution. In practice, the test harness exposes the patch *text* to the task-specific tests, enabling purely diff-based verification.

Static Analysis. Before running task-specific assertions, we perform lightweight static checks on the diff text. This includes verifying unified diff structure (e.g., `diff -git` headers), rejecting empty or near-empty patches, and ensuring that added lines contain meaningful code changes rather than only whitespace or comments. We also check whether the patch touches relevant files using flexible path patterns (e.g., accepting file moves/renames), and apply basic language-agnostic sanity checks to filter malformed submissions early.

Diff-Based Intent Tests. Direct runtime evaluation for mobile applications is challenging to scale. Unit tests are ill-suited for validating visual correctness, while end-to-end UI testing introduces substantial compilation overhead and environmental nondeterminism. To address these constraints, SWE-Bench Mobile adopts a **diff-based evaluation** strategy: our `pytest` suites inspect the patch diff and verify *structural intent* and *architectural compliance*. This allows us to evaluate high-level architectural decisions and requirement compliance at scale. Tests are constructed from the PRD and a human reference patch, emphasizing:

- **Goal-oriented checks**: verifying modification patterns (the "what") rather than exact code shape.

Table 2: Comparison with existing benchmarks.

Benchmark	Multi-Modal	Codebase	Language
HumanEval	✗	None	Python
MBPP	✗	None	Python
SWE-Bench	✗	Medium	Python
SWE-Bench Mobile	✓	Large	Swift/ObjC

- **Feature entry points**: checking integration surfaces (e.g., routing, hooks).
- **Removal of blocking behavior**: ensuring constraints or legacy guards are lifted.
- **Cohesion across files**: verifying related edits across modules.
- **Semantics-aware matching**: using flexible pattern matching to accommodate alternative naming.

Batch Reporting and Error Analysis. Beyond pass/fail decisions, our evaluator produces both task-level and test-case-level summaries. For large-scale runs, we classify failures into coarse categories (e.g., missing critical file edits, missing UI components, empty patches). This analysis provides interpretable diagnostics of common agent failure modes and supports systematic iteration on prompts and agent scaffolding.

Metrics. We report two complementary metrics. **Task Success Rate** is the percentage of tasks where *all* tests pass, representing the strict standard for a completed feature. **Test Pass Rate** is the percentage of individual test cases passed, which reveals partial progress even when the full task is not completed. The gap between these metrics reveals how often agents make partial progress without fully completing tasks.

2.6 Comparison with Existing Benchmarks

Table 2 compares SWE-Bench Mobile with existing coding benchmarks. SWE-Bench Mobile distinguishes itself by being multi-modal, including PRDs and Figma designs rather than just code or text descriptions. It operates on a large-scale codebase (~5GB), significantly larger than the individual repositories or snippets used in other benchmarks. Furthermore, it targets mixed Swift/Objective-C, which is under-represented in training data compared to Python, and focuses on feature implementation rather than bug fixing.

3 Experiments

We evaluate leading coding agents on SWE-Bench Mobile to answer several key research questions. First, we investigate how state-of-the-art coding agents perform on industry-level mobile development tasks (**RQ1**). Second, we analyze how task complexity affects agent performance (**RQ2**). Third, we examine the cost-performance trade-off (**RQ3**). Fourth, we assess the robustness of agent results across multiple runs (**RQ4**). Finally, we explore how prompt engineering affects performance (**RQ5**).

3.1 Experimental Setup

Agents and Models. We evaluate four coding agents spanning commercial and open-source systems: **Cursor**, an AI-powered code editor with an agent mode; **Codex**, OpenAI's coding agent CLI; **Claude Code**, Anthropic's coding agent CLI; and **OpenCode**, an open-source coding agent. We test these agents with multiple backbone models including Claude Opus 4.5, Claude Sonnet 4.5, Claude Haiku, GLM-4.6, GLM-4.7, GPT-5, GPT-5.1, GPT-5.2, and Gemini 3 Pro, yielding 22 agent-model configurations in total.

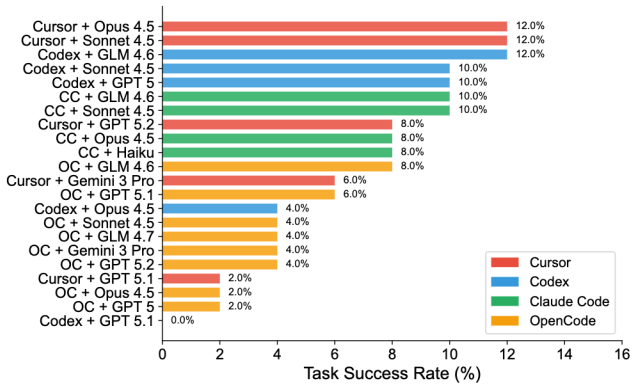


Figure 4: Task Success Rate across all configurations. Best performance is 12%, achieved by Cursor + Opus/Sonnet and Codex + GLM.

Metrics. We report two primary metrics: **Task Success Rate**, which is the percentage of tasks where *all* test cases pass, and **Test Pass Rate**, which is the percentage of individual test cases passed. All rates are computed with a fixed denominator of 50 tasks and 449 test cases. When an agent fails to produce a patch for a task (e.g., due to timeout or error), the missing patch is counted as failing all associated tests.

3.2 Main Results (RQ1)

Figure 4 presents the main experimental results across all agent-model configurations.

Key Findings. Our evaluation reveals a generally low overall performance, with even the best agents solving only 12% of tasks. This indicates a significant gap between current capabilities and industrial requirements. However, the Test Pass Rate (up to 28.1%) is much higher than the Task Success Rate (12%), indicating that agents often make partial progress but fail to complete tasks fully. Notably, we find that the choice of agent matters significantly: the same model (Opus 4.5) achieves 12% on Cursor but only 2% on OpenCode, a 6× difference. Commercial agents consistently outperform the open-source OpenCode agent: the best OpenCode configuration (GLM-4.6, 8%) trails the best commercial configuration (12%) by 4 percentage points.

3.3 Task Complexity Analysis (RQ2)

We analyze how task complexity affects agent performance. Figure 5 shows the relationship between task complexity (measured by number of files modified and patch size) and success rate.

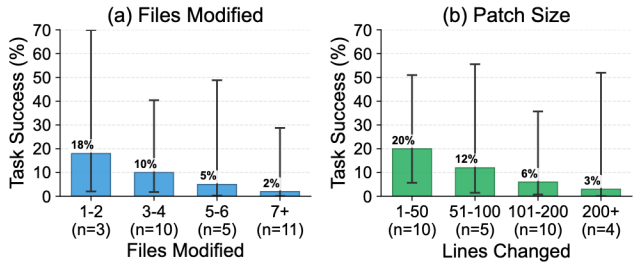


Figure 5: Performance decreases sharply with task complexity. (a) Tasks requiring 1-2 file modifications have 18% success rate vs. 2% for 7+ files. (b) Small patches (<50 lines) achieve 20% success vs. 3% for large patches (>200 lines). Error bars show 95% confidence intervals based on binomial proportions.

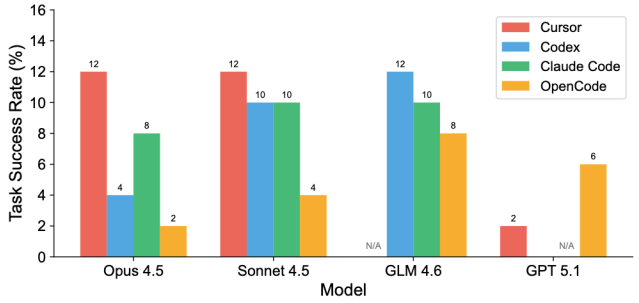


Figure 6: Same model, different agents: Opus 4.5 achieves 12% on Cursor but only 2% on OpenCode—a 6× gap. Commercial agents consistently outperform the open-source alternative.

Key Findings. Performance drops sharply as complexity increases. The success rate drops from 18% for tasks requiring 1-2 file modifications to just 2% for tasks requiring 7+ files, suggesting that agents struggle with cross-file reasoning. Similarly, larger patches correlate with lower success, indicating difficulty with complex implementations.

3.4 Model Comparison Across Agents

A surprising finding is that **the same model performs very differently across agents**. Figure 6 shows this comparison across all four agents.

Implications. This finding suggests that agent scaffolding (tool use, context management, iteration strategy) is as important as the underlying model capability. The performance gap between commercial agents (Cursor, Codex, Claude Code) and the open-source OpenCode is substantial across all models, suggesting that years of engineering investment in tool integration, context management, and iterative refinement provide significant advantages. Practitioners should evaluate agents holistically rather than focusing solely on model benchmarks.

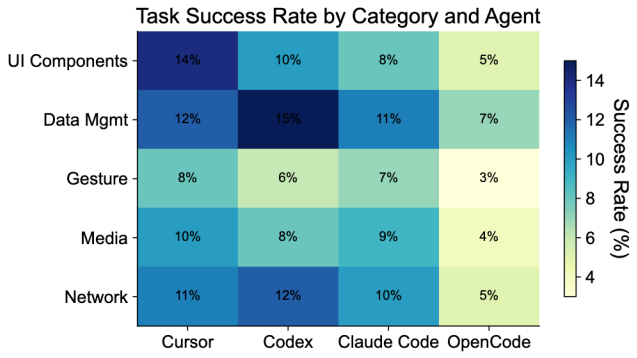


Figure 7: Task Success Rate by Category and Agent. Agents generally perform better on Data Management tasks but struggle with Gesture & Interaction and Media tasks, which require complex multi-modal reasoning.

Table 3: Cost and time comparison across all agents. Best value in each column is bold. OpenCode costs are reported via OpenRouter API billing. CC denotes Claude Code; OC denotes OpenCode.

Agent	Model	Cost ($/task)	Time (min)
Cursor	Opus 4.5	3.50	15.0
Cursor	Sonnet 4.5	2.00	14.2
Codex	GLM-4.6	1.30	13.3
Codex	Sonnet 4.5	2.50	12.5
CC	GLM-4.6	1.30	11.7
CC	Sonnet 4.5	2.00	13.3
CC	Opus 4.5	4.00	15.0
CC	Haiku	0.50	8.3
OC	Opus 4.5	9.33	8.2
OC	Sonnet 4.5	3.50	11.1
OC	GLM-4.6	0.13	32.5
OC	GLM-4.7	0.49	52.1
OC	GPT-5	0.18	9.8
OC	GPT-5.1	**0.02**	**2.0**
OC	GPT-5.2	0.04	10.9
OC	Gemini 3 Pro	0.03	8.9

3.5 Performance by Task Category

We analyze how agents perform across different task categories. Figure 7 shows the success rate breakdown.

3.6 Cost and Time Analysis (RQ3)

Table 3 presents the cost and time metrics for each configuration. We measure API cost per task and average execution time.

Key Findings. Among commercial agents, Codex + GLM-4.6 offers the best value, achieving 12% success at only $1.30/task—the same success rate as Cursor + Opus 4.5 but at less than half the cost ($3.50/task). OpenCode exhibits a striking cost–time tradeoff: it is dramatically cheaper (GLM-4.6 at $0.13/task vs. $1.30 for Codex/CC), but GLM models run much slower (32–52 min vs. 11–13 min). OpenCode + Opus 4.5 is the most expensive configuration at

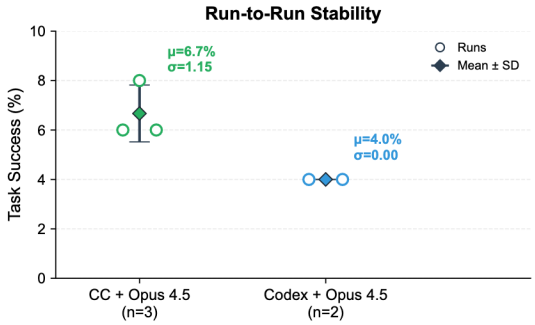


Figure 8: Result stability across multiple runs. Error bars indicate standard deviation. While Claude Code shows moderate variance (σ=1.15%), the absolute fluctuation is small (±1 task), indicating that agent performance is relatively stable.

Table 4: Prompt ablation results. Best and worst highlighted.

Prompt Strategy	Task (%)	Test (%)
Defensive Programming	**10.0**	**26.7**
Quality Focused	10.0	26.3
Example Driven	10.0	23.4
Chain of Thought	10.0	21.8
Baseline	10.0	19.3
Explicit Instructions	8.0	17.8
Figma Emphasis	8.0	18.0
Test Driven	6.0	22.0
Detailed Role	4.0	20.7
Structured Checklist	4.0	20.7
Context Rich	4.0	22.7
Comprehensive	4.0	22.7

$9.33/task yet achieves only 2% success, while OpenCode + GPT-5.1 is the cheapest at $0.02/task but completes tasks in only 2 minutes on average—likely because it fails quickly on most tasks (6% success, 7.1% test pass rate).

3.7 Robustness Analysis (RQ4)

To assess result stability, we run selected configurations multiple times. Figure 8 shows the variance across runs.

Observations. We observe moderate variance for Claude Code + Opus 4.5, with scores of 6%, 8%, and 6% across 3 runs (μ=6.7%, σ=1.15%). In contrast, Codex + Opus 4.5 is perfectly stable at 4% across runs.

3.8 Prompt Engineering (RQ5)

We conduct a systematic ablation study with 12 prompt variants using Claude Code + GLM-4.6. Table 4 shows the results.

Key Findings. The "Defensive Programming" prompt strategy performs best, improving the Test Pass Rate by 7.4% over the baseline (19.3% → 26.7%) while maintaining the same Task Success Rate (10.0%). This indicates that while both prompts complete the same number of tasks fully, Defensive Programming handles edge cases better in partially-completed tasks, passing significantly more individual test cases. This suggests that emphasizing defensive coding practices helps agents avoid common pitfalls even when they cannot complete all requirements. Interestingly, complexity appears to hurt performance; overly detailed prompts reduce Task Success from 10.0% to 4.0%. Overall, prompts focusing on code quality outperform those emphasizing workflow.

3.9 Error Analysis

We categorize failure modes across all experiments by analyzing test failure messages from the best-performing agents; because a single failed task often exhibits several modes, the percentages below are not mutually exclusive. The most critical failure pattern is **Missing Feature Flags** (54%), where agents implement core functionality but fail to add proper feature toggles or experiment flags—a standard practice in production mobile development for gradual rollout and A/B testing. **Missing Data Models** (22%) occurs when agents fail to create or update data structures required by the PRD. **Missing Files** (11-15%) represents cases where agents identify some but not all required files to modify. **Missing UI Components** (11-15%) captures failures to implement specific UI elements like buttons, labels, or views. **Missing Required Methods** (9%) reflects incomplete class implementations. While **Incomplete Multi-File Implementation** affects only 4-7% of tasks, it disproportionately impacts complex features requiring coordination across 5+ files. The dominance of feature flag failures highlights a gap between agents' code generation capabilities and their understanding of production deployment practices.

4 Discussion and Conclusion

Our evaluation reveals a significant gap between current agent capabilities and the demands of industrial mobile development, with the best configurations achieving only a 12% success rate. This shortfall, primarily driven by gaps in production practices (e.g., missing feature flags) and weak cross-file coordination, underscores that autonomous software engineering remains an open challenge.

Implications. For **practitioners**, our results suggest that agents should currently be viewed as "copilots" requiring human oversight rather than autonomous developers. The high variance in performance across agents for the same model (e.g., Cursor 12% vs. OpenCode 2% for Opus 4.5) highlights the critical role of agent scaffolding—practitioners should evaluate the complete system, not just the underlying model. The consistent gap between commercial and open-source agents suggests that engineering investment in tool integration and context management provides significant practical value. Furthermore, cost-effective models like GLM-4.6 can match the performance of expensive frontier models when paired with effective agent frameworks, offering a viable path for scalable adoption. For **researchers**, the sharp performance drop on complex, multi-file tasks (18% vs. 2%) points to a need for better code context retrieval and graph-based reasoning. Our failure analysis points to a more specific gap: the dominant errors are production-practice omissions—54% of failures involve missing feature flags and 22% missing data models—rather than algorithmic mistakes, calling for

agents that internalize deployment conventions and ground PRD requirements into the correct files and modules.

Future Work. We plan to expand SWE-Bench Mobile to include Android (Kotlin) tasks for cross-platform comparison and integrate simulator-based runtime evaluation to verify UI dynamics. We also aim to evaluate additional open-source agents like OpenHands and SWE-Agent, and open-weight models like Qwen-Coder, to further broaden the benchmark's coverage.

In conclusion, SWE-Bench Mobile provides a rigorous testbed for the next generation of coding agents. While current performance is modest, the benchmark offers a clear roadmap for advancing agents from simple script generation to complex, industry-level software development.

5 Related Work

5.1 Code Generation Benchmarks

Early benchmarks for code generation focused on algorithmic problem-solving. **HumanEval** [3] introduced 164 hand-crafted Python programming problems with unit tests, becoming a standard evaluation for code LLMs. **MBPP** [2] expanded this with 974 crowd-sourced problems. While influential, these benchmarks test isolated function generation rather than realistic software engineering.

SWE-Bench [7] marked a significant advance by evaluating agents on real GitHub issues from popular Python repositories. Agents must understand issue descriptions, navigate codebases, and generate patches that pass existing tests. The benchmark has since evolved into a family of tasks, including **SWE-bench Multimodal** [15], which incorporates visual elements such as screenshots and diagrams to test visual software domains; **SWE-bench Multilingual** [16, 17], which expands evaluation to 9 programming languages beyond Python; and **SWE-bench Pro** [5], which introduces longer-horizon instances and includes proprietary/commercial codebases. Even with these extensions, many existing benchmarks still derive tasks from GitHub issue and pull-request artifacts, which more often emphasize bug fixing and localized improvements than new feature implementation from high-level specifications.

Other benchmarks target specific domains: **DS-1000** [8] for data science, **ODEX** [11] for open-domain execution, and **ClassEval** [6] for class-level generation. **DevBench** [9] evaluates repository-level coding but still focuses on Python.

SWE-Bench Mobile differs from these benchmarks in several key aspects: (1) multi-modal inputs including PRDs and Figma designs, (2) a large-scale production codebase (approx. 5GB), (3) mixed Swift/Objective-C target languages, and (4) feature implementation rather than bug fixing.

5.2 Coding Agents

The emergence of powerful LLMs has enabled a new generation of autonomous coding agents. These systems go beyond simple code completion to perform multi-step reasoning, tool use, and iterative refinement.

Commercial agents include GitHub Copilot (Microsoft), Cursor (Anysphere), Claude Code (Anthropic), and Codex CLI (OpenAI). These agents integrate with development environments and can navigate codebases, run tests, and iterate on solutions.

Open-source agents have emerged as alternatives. OpenCode provides a terminal-based coding agent supporting multiple LLM backends. SWE-Agent [14] introduces an agent-computer interface optimized for software engineering. AutoCodeRover [19] combines code search with LLM reasoning. Agentless [13] shows that simpler approaches without complex agent loops can be competitive. CodeAgent [18] uses a repository-level code graph for navigation.

Our work provides a challenging benchmark for evaluating both commercial and open-source agents on industry-level tasks, revealing significant gaps in current capabilities and the importance of agent scaffolding.

5.3 Multi-Modal Code Understanding

Recent work has explored combining visual and textual information for code-related tasks. **Design2Code** [10] evaluates generating code from webpage screenshots. **Screenshot2Code** systems convert UI designs to implementation.

SWE-Bench Mobile extends this direction by incorporating Figma designs as part of the input specification, requiring agents to reason about visual layouts alongside textual requirements.

5.4 Prompt Engineering for Code

Prompt engineering significantly impacts LLM performance on coding tasks. Chain-of-thought prompting [12] improves reasoning. Self-debugging [4] enables iterative refinement. Structured prompts with role definitions and examples often outperform simple instructions.

Our ablation study (Section 3.8) systematically evaluates 12 prompt strategies, finding that "Defensive Programming" prompts emphasizing edge cases outperform both simple baselines and complex multi-step prompts.

Limitations

SWE-Bench Mobile focuses on a single production iOS codebase from XiaoHongShu, which ensures depth and realism but limits generalization to other mobile platforms (Android, cross-platform frameworks) and programming paradigms. Our evaluation uses text-based diff inspection rather than runtime execution, which means we validate structural correctness and architectural compliance but cannot detect issues that only manifest during runtime interactions or on specific devices. Additionally, API costs reported are based on pricing at experiment time and may vary with different prompting strategies or model updates.

Our prompt ablation study covers one agent-model configuration (Claude Code + GLM-4.6) and 12 prompt variants. While this provides insights into prompt sensitivity, different models may respond differently to these strategies. The benchmark's 50 tasks, while derived from real product development, represent a snapshot of mobile development challenges and may not cover all possible feature types or edge cases encountered in production.

Ethics Statement

The tasks and codebase in SWE-Bench Mobile are derived from XiaoHongShu Inc. with explicit permission for research use. The

codebase snapshot excludes sensitive credentials and business logic. Human validation was performed by the authors and XiaoHongShu engineers; no crowdworkers were employed.

Our work evaluates AI agents for software engineering tasks. Current performance (12% task success rate) indicates that human oversight remains essential. We view these agents as assistive tools rather than replacements for human developers. Practitioners should use comprehensive testing and code review when deploying AI-generated code, as emphasized by our benchmark's evaluation approach.

References

[1] Atlassian. 2024. How to Write a Product Requirements Document (PRD). https://www.atlassian.com/agile/product-management/requirements.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[4] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. *arXiv preprint arXiv:2304.05128* (2023).

[5] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? *arXiv preprint arXiv:2509.16941* (2025).

[6] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation. In *International Conference on Machine Learning*.

[7] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *The Twelfth International Conference on Learning Representations*.

[8] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In *International Conference on Machine Learning*.

[9] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Xiong, and Karthik Narasimhan. 2024. DevBench: A Comprehensive Benchmark for Software Development. *arXiv preprint arXiv:2403.08604* (2024).

[10] Chenglei Si, Yanzhe Li, Zhengyuan Jiang, Xinyang Liu, Zheng Lu, Yuqing Jiang, Yong Liu, Yu Wang, Yujiu Yuan, Lydia Liu, et al. 2024. Design2Code: How Far Are We From Automating Front-End Engineering? *arXiv preprint arXiv:2403.03163* (2024).

[11] Zhiruo Wang, Shuyan Zhou, Daniel Fried, and Graham Neubig. 2022. Execution-Based Evaluation for Open-Domain Code Generation. *arXiv preprint arXiv:2212.10481* (2022).

[12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[13] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489* (2024).

[14] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793* (2024).

[15] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ofir Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. 2025. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?. In *The Thirteenth International Conference on Learning Representations*.

[16] John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling Data for Software Engineering Agents. *arXiv preprint arXiv:2504.21798* (2025).

[17] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. arXiv:2504.02605 [cs.SE] https://arxiv.org/abs/2504.02605

[18] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. *arXiv preprint arXiv:2401.07339* (2024).

[19] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. *arXiv preprint arXiv:2404.05427* (2024).

A Complete Experimental Results

Additional task examples, prompt templates, design mockups, and qualitative case studies are omitted from the proceedings version to satisfy the page limit. The public leaderboard is available at https://swebenchmobile.com.

B Reproducibility

Benchmark Construction. Our construction pipeline is platform-agnostic and can be reused by any organization with a production codebase: (1) select real product requirements with self-contained PRDs; (2) collect the associated design specifications (Figma, Sketch, etc.); (3) have engineers produce a reference patch; (4) write a diff-based `pytest` suite that verifies structural intent via semantic pattern matching; and (5) run multi-round human validation.

Environment. All experiments were conducted on macOS 14.x with:

- Cursor: v2.3 with Agent mode enabled
- Codex: OpenAI Codex CLI v0.77.0
- Claude Code: Anthropic Claude Code CLI v2.1.37
- OpenCode: v1.1.44 (open-source coding agent)

Model API Configuration.

For reproducibility, we specify the exact API endpoints and configurations used:

- **GPT Models (GPT-5, GPT-5.1, GPT-5.2)**: Accessed via Microsoft Azure OpenAI API with default temperature and top-p settings
- **Claude Models (Opus 4.5, Sonnet 4.5, Haiku)**: Accessed via Google Vertex AI API for Anthropic models
- **Gemini 3 Pro**: Accessed via Google Vertex AI API with standard configuration
- **GLM Models (GLM-4.6, GLM-4.7)**: Used GLM Coding Plan with default agent scaffolding

Multi-Modal Input Handling.

To handle Figma designs and reference images, we configured Model Context Protocol (MCP) integrations:

- **Vision-capable models (GPT, Claude, Gemini)**: Used official Figma MCP to directly access design specifications
- **GLM Models**: Since GLM-4.6 is not a native vision model, we used the official GLM Vision MCP to process images and Figma designs, converting visual inputs into structured descriptions for the text-only model

Evaluation Pipeline.

(1) Load generated patch file as text

Table 5: Complete evaluation results on SWE-Bench Mobile. Task Success measures the percentage of tasks where all test cases pass (out of 50 tasks). Test Pass measures the percentage of individual test cases passed (out of 449 tests). Best results per agent in bold.

Agent	Model	Task Success (%)	Test Pass (%)	Cost ($/task)	Time (min)
Cursor	Claude Opus 4.5	**12.0**	**28.1**	3.50	15.0
	Claude Sonnet 4.5	**12.0**	26.7	2.00	14.2
	GPT-5.2	8.0	27.4	1.80	20.0
	Gemini 3 Pro	6.0	23.2	1.00	12.5
	GPT-5.1	2.0	19.6	1.10	14.2
Codex	GLM-4.6	**12.0**	19.6	1.30	13.3
	Claude Sonnet 4.5	10.0	**28.1**	2.50	12.5
	GPT-5	10.0	21.4	1.50	10.0
	Claude Opus 4.5	4.0	20.7	3.50	14.2
	GPT-5.1	0.0	7.1	1.00	13.3
Claude Code	GLM-4.6	**10.0**	**26.7**	1.30	11.7
	Claude Sonnet 4.5	10.0	24.7	2.00	13.3
	Claude Opus 4.5	8.0	21.8	4.00	15.0
	Claude Haiku	8.0	18.3	0.50	8.3
OpenCode	GLM-4.6	**8.0**	**17.8**	0.13	32.5
	GPT-5.1	6.0	7.1	0.02	2.0
	Claude Sonnet 4.5	4.0	14.7	3.50	11.1
	GLM-4.7	4.0	14.3	0.49	52.1
	Gemini 3 Pro	4.0	13.4	0.03	8.9
	GPT-5.2	4.0	12.0	0.04	10.9
	Claude Opus 4.5	2.0	12.0	9.33	8.2
	GPT-5	2.0	12.0	0.18	9.8

(2) Run task-specific pytest test suite (tests inspect the patch diff text using pattern matching and structural analysis)

(3) Record pass/fail status for each test case

(4) Aggregate results across all 50 tasks

Availability and Hosted Evaluation. The SWE-Bench Mobile benchmark is derived from a proprietary production codebase with permission from XiaoHongShu Inc. Due to the confidential nature of the source code and product requirements, the full dataset cannot be publicly released. We view this constraint as a feature rather than a limitation: by keeping the test set private, we *eliminate the risk of data contamination*—a well-known issue with public benchmarks where test instances may leak into LLM training corpora [7].

SWE-Bench Mobile is designed as a *standardized evaluation platform for coding agent providers and foundation model vendors*. We host a public leaderboard at https://swebenchmobile.com where agent companies (e.g., Cursor, Codex, Claude Code) and model providers (e.g., OpenAI, Anthropic, Google, Zhipu AI) can submit their systems for evaluation against our held-out industrial test suite. This provides an objective, contamination-free comparison on real-world mobile development tasks that complements existing Python-centric benchmarks. Interested parties can contact the authors through the leaderboard website for hosted evaluation details.

C Key Insights from Agent Failures

Based on detailed examination of 50+ agent-generated patches across all 4 agents, we identify several patterns:

Success Predictors.

- **Well-scoped changes:** Tasks requiring 1-3 files with clear boundaries (e.g., creating new data models, updating configuration logic)
- **Clear architectural guidance:** PRDs that specify class names, method signatures, or reference existing code patterns
- **Isolated modules:** Features in self-contained modules with minimal cross-module dependencies
- **Existing patterns:** Tasks that resemble code already in the codebase that agents can reference

Failure Predictors.

- **Distributed logic:** Changes spanning multiple architectural layers (Model-View-Controller-Manager)
- **Implicit requirements:** PRDs using phrases like "all scenarios" or "everywhere" without explicit file lists